



Programa Certified AI-LLM Integration Engineer

Curso de Fundamentos de LLMs y APIs

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 23 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialización en MLOps de la Universidad de Duke



arojaspa@outlook.com



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373

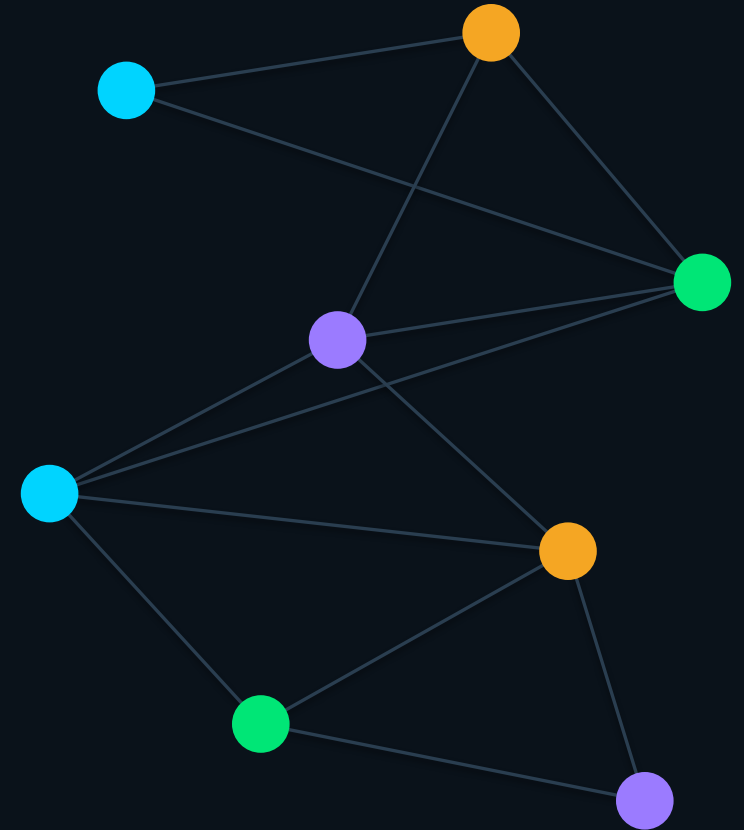


BSG INSTITUTE · CAPÍTULO 1: CONCEPTOS FUNDAMENTALES

Fundamentos de Arquitectura LLM

Sesión 1 · Introducción a las arquitecturas LLM

Sesión 2 · Tipos de modelos y servicios



Diseño e implementación de soluciones con LLM para el mercado LATAM

Dos sesiones, una idea: entender antes de construir

SESIÓN 1

Introducción a las arquitecturas LLM

- Origen y evolución: de ELIZA al Transformer
- Anatomía: tokens, embeddings, atención
- Entrenamiento e inferencia
- Limitaciones y riesgos
- Relevancia e impacto en la industria

SESIÓN 2

Tipos de modelos y servicios

- Decoder-only, encoder-only, encoder-decoder
- Nube, on-premise e híbrido
- Pesos abiertos vs. cerrados
- Casos de uso LATAM
- Costos y factores de selección

Al terminar esta sesión usted podrá...

Recordar

los hitos que llevaron de los modelos estadísticos a los LLM actuales

Comprender

cómo un LLM genera texto prediciendo el siguiente token

Comprender

los bloques de un Transformer: tokens, embeddings, atención y salida

Aplicar

el consumo de un LLM local vía API e interpretar tokens y latencia

Analizar

cómo tokens, contexto y temperatura afectan costo, calidad y riesgo

TechCorp Latinoamérica

Empresa manufacturera con sedes en Bogotá, Lima y Ciudad de México. Quiere incorporar LLM en su operación sin comprometer la seguridad de sus datos ni su presupuesto.

PREGUNTA QUE GUIARÁ EL CURSO

¿Qué modelo, desplegado dónde y a qué costo, resuelve cada necesidad con la calidad y el riesgo aceptables?

1

Mesa de ayuda de TI

Responder 3.000
consultas/mes de
colaboradores

2

Manuales técnicos

Consultar 20.000 páginas en
lenguaje natural

3

Calidad

Clasificar reportes de no
conformidad

4

Gobierno

Cumplir protección de datos
en 3 países

01

Introducción a las arquitecturas LLM

Origen y evolución · Cómo funciona un LLM · Relevancia e impacto en la industria

¿Qué ocurre cuando escribe en un chatbot?

“Explícame qué es un crédito de libranza.”

- ? ¿Busca en internet?
- ? ¿Consulta una base de datos de respuestas?
- ? ¿“Entiende” como una persona?
- ? ¿Copia textos que memorizó?

LA RESPUESTA (AL FINAL DE LA SESIÓN)

Calcula, token a token, la continuación más probable del texto, usando patrones estadísticos aprendidos de billones de palabras.

No consulta, no busca y no “sabe”: genera. Por eso puede ser brillante... o equivocarse con total seguridad.

¿Qué es un modelo de lenguaje?

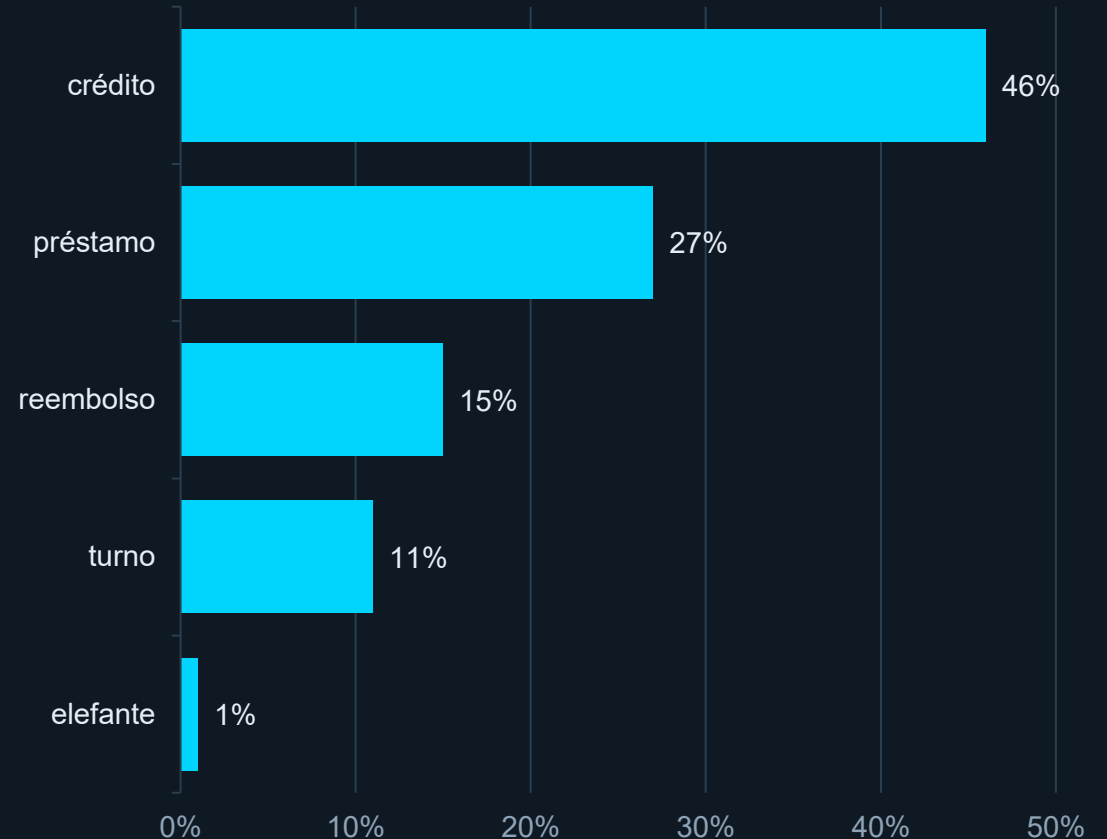
Un modelo de lenguaje estima la probabilidad de la siguiente palabra (token) dado el contexto anterior.

$$P(\text{siguiente token} \mid \text{contexto})$$

Un LLM (Large Language Model) es lo mismo, pero:

- **Grande en parámetros:** miles de millones de pesos aprendidos
- **Grande en datos:** billones de tokens de entrenamiento
- **Basado en Transformer:** procesa contexto largo con atención

“El cliente solicita un ____”



Origen: de las reglas a las redes neuronales (1950–2014)



Evolución: la era del Transformer (2017–2026)



Tres fuerzas explican la evolución

1 Representación

De símbolos aislados a vectores (embeddings) donde la distancia refleja el significado.

2 Arquitectura

De leer palabra por palabra (RNN) a relacionar toda la secuencia en paralelo con atención (Transformer).

3 Escala + alineamiento

Más datos, parámetros y cómputo → nuevas capacidades. El ajuste con instrucciones y preferencias humanas las vuelve útiles.

Idea clave: la tarea no cambió en 70 años (predecir la siguiente palabra); cambiaron la representación, la arquitectura y la escala con que se aprende.

De palabras a vectores: el significado como geometría

Proyección 2D ilustrativa de un espacio de embeddings



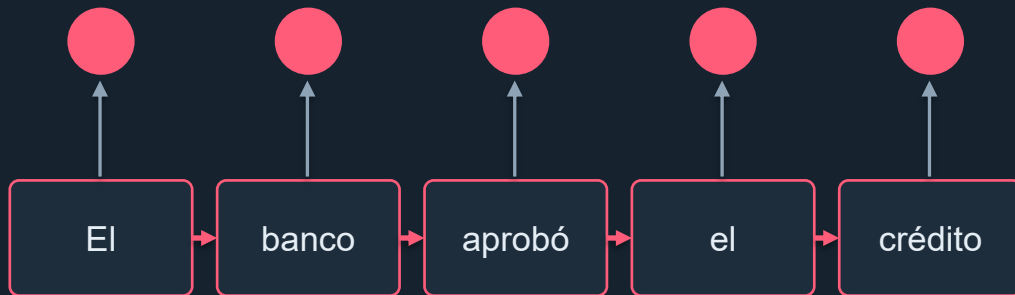
- **Embedding:** vector de cientos o miles de números que representa un texto.
- **Cercanía** = similitud de significado.
- **Relaciones como direcciones:** rey – hombre + mujer $\approx$ reina.
- **Base de búsqueda semántica,** clasificación y RAG.

```
"crédito" → [0.12, -0.87,  
             0.33, ..., 0.05] (768 dim.)
```

El salto de 2017: de leer en fila a mirar todo a la vez

RNN / LSTM (hasta 2017)

estado oculto → → → →



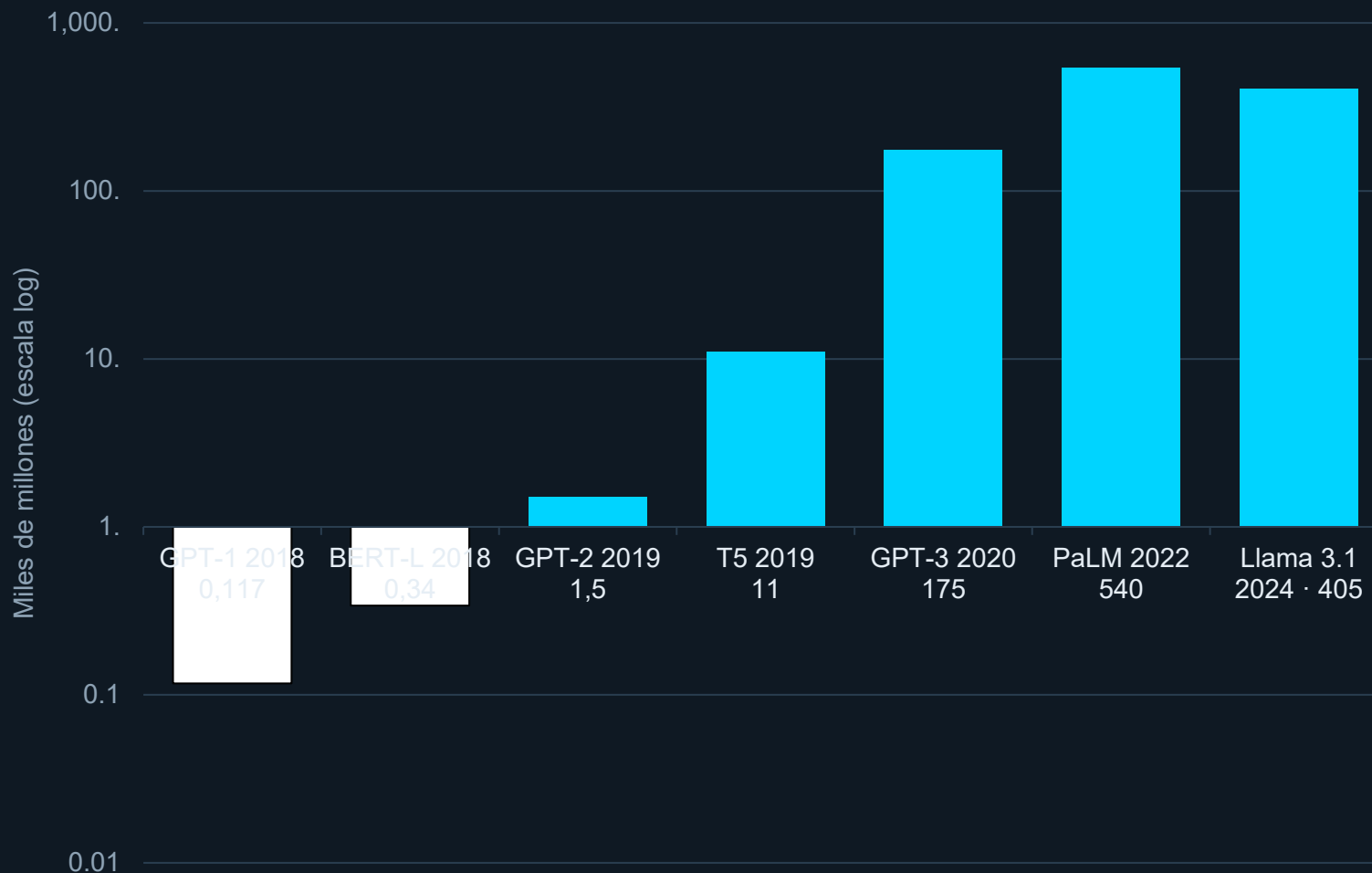
Secuencial: cada paso depende del anterior. Lento de entrenar y olvida el contexto lejano.

Transformer (2017 →)



Paralelo: cada token atiende a todos los demás. Escala en GPU y captura dependencias largas.

La escala: crecimiento de los modelos (parámetros)



Leyes de escala

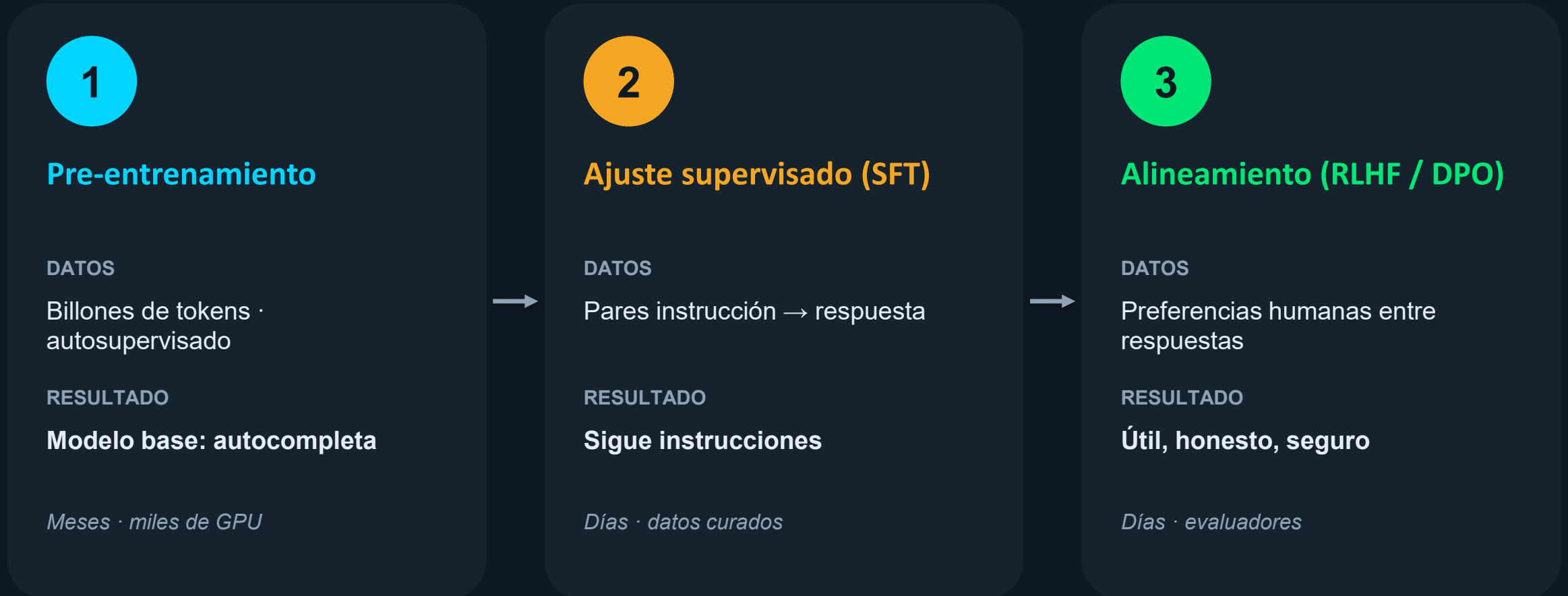
El error baja de forma predecible al aumentar parámetros, datos y cómputo.



Para el arquitecto

Más parámetros = más memoria, más latencia y más costo por token.

De modelo base a asistente: las tres etapas de entrenamiento



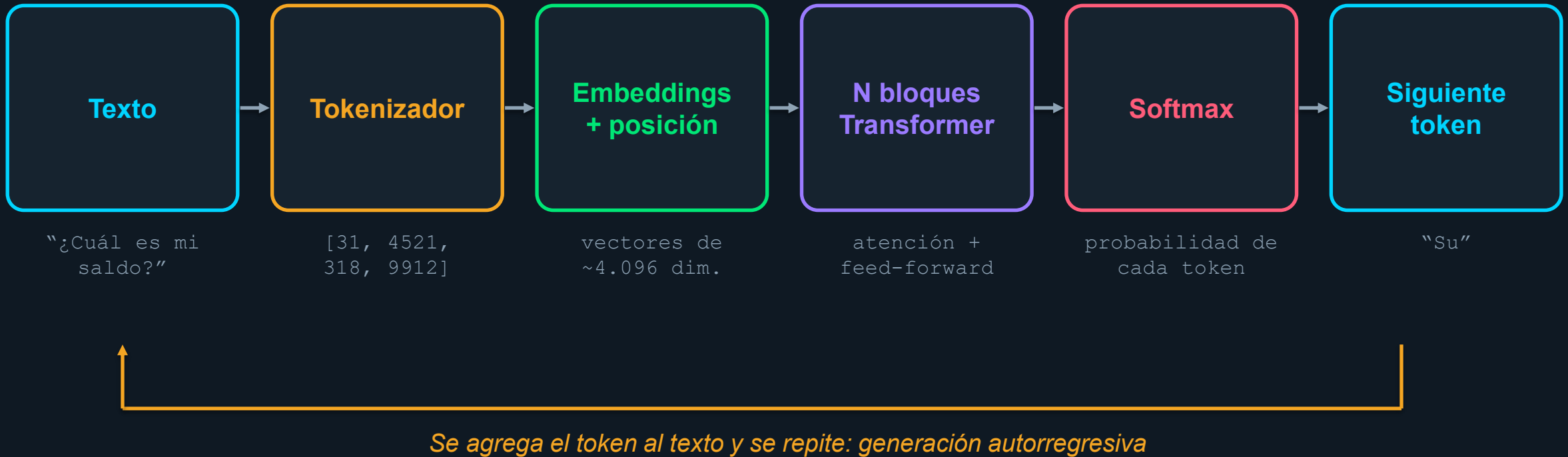
Las empresas casi nunca hacen la etapa 1: consumen modelos ya entrenados y los adaptan con prompts, RAG o ajuste fino.

1.2

¿Cómo funciona un LLM por dentro?

El recorrido de un prompt: tokens → embeddings → atención → probabilidades → texto

El recorrido de un prompt dentro del modelo



Generar 500 tokens = 500 pasadas por el modelo. Por eso la salida es más lenta y más cara que la entrada.

Tokenización: la unidad de cómputo, de contexto y de cobro



Ejemplo ilustrativo: 4 palabras → 9 tokens (“.” = espacio)

Texto	Palabras	Tokens*
“¿Cuál es el saldo de mi cuenta?”	7	≈ 9
“What is my account balance?”	5	≈ 6
“1.234.567,89”	1	≈ 7
def suma(a, b): return a+b	—	≈ 11

* Valores aproximados; dependen del tokenizador de cada modelo.

1,3–1,6

tokens por palabra en español

~700

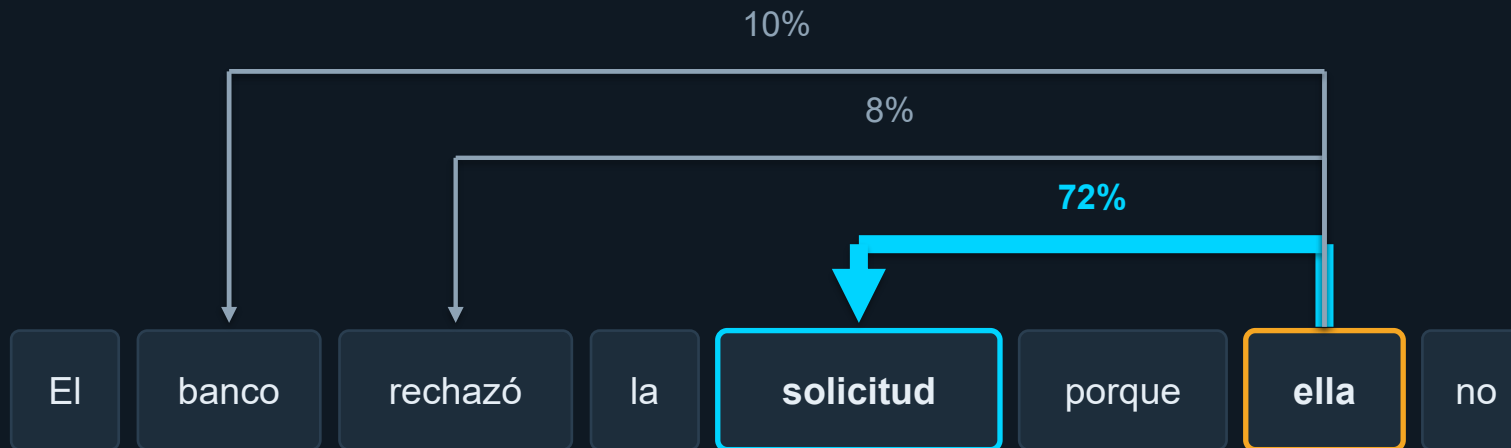
tokens por página de texto

100–200 mil

tokens en el vocabulario

Auto-atención: cada token decide a quién prestar atención

Al procesar “ella”, el modelo asigna más peso a “solicitud” (pesos ilustrativos).



Costo: cada token se compara con todos los demás → el cómputo crece aproximadamente con el cuadrado de la longitud del contexto.

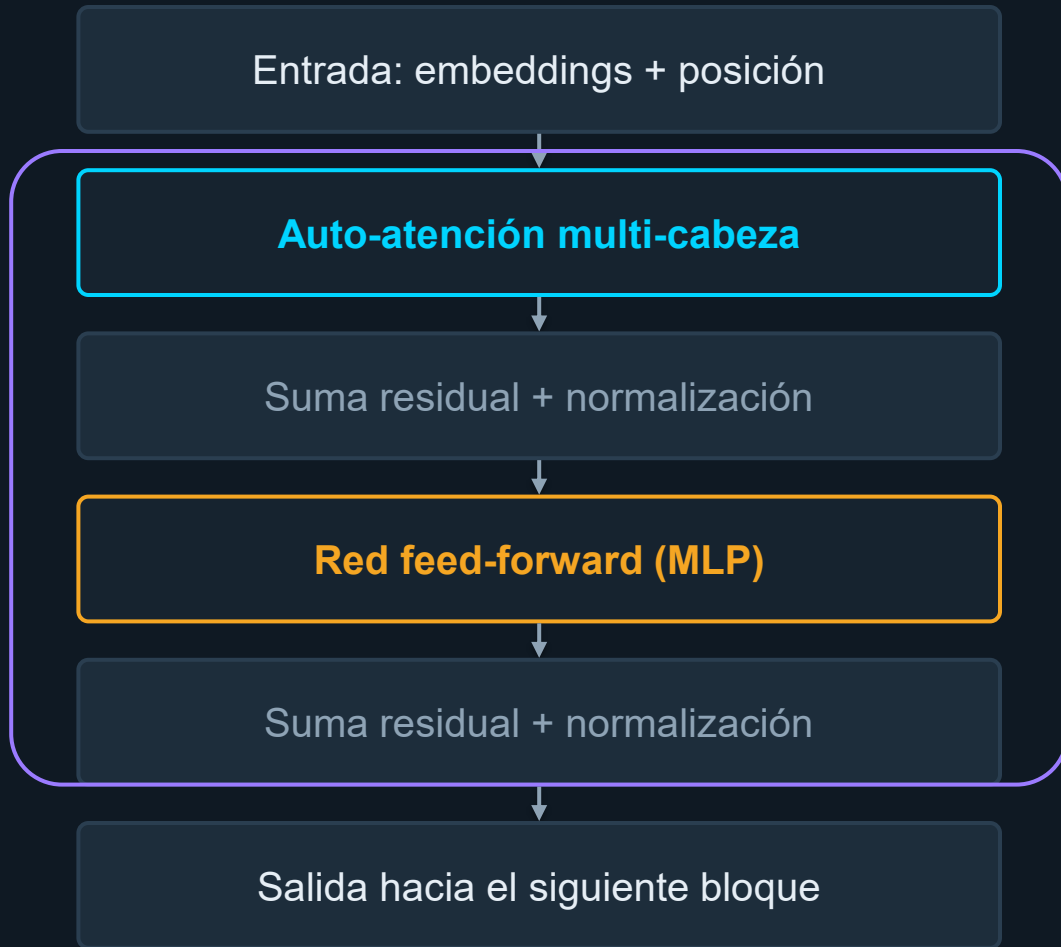
CÓMO SE CALCULA

- **Q · consulta:** ¿qué busco?
- **K · clave:** ¿qué ofrezco?
- **V · valor:** ¿qué información apporto?

$$\text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

Multi-cabeza: varias atenciones en paralelo, cada una especializada en un tipo de relación.

El bloque Transformer, apilado N veces



× N

32–120+

capas en LLM actuales

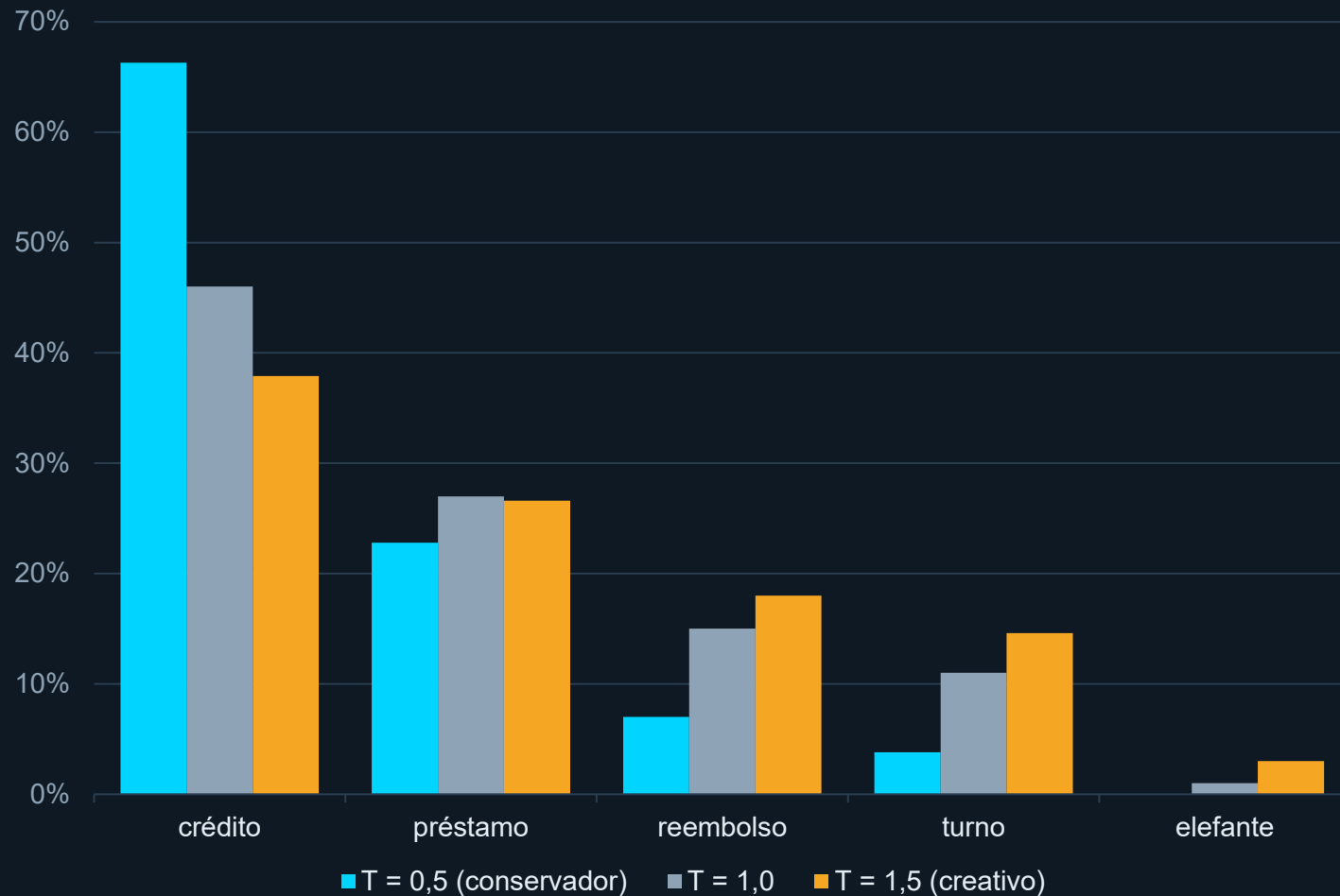
~16 GB

pesos de un modelo de 8B en 16 bits

~2 GB

llama3.2:3b cuantizado a 4 bits

Del vector a la palabra: softmax, muestreo y temperatura



Temperatura baja

Respuestas consistentes. Ideal para extracción, clasificación, SQL y código.



Temperatura alta

Más variedad y creatividad... y más riesgo de incoherencias o invenciones.

Parámetros de decodificación: perillas de diseño



Temperatura

Aleatoriedad del muestreo

0–0,3 extracción · 0,7 chat · 1+ ideas



Top-p

Solo tokens del “núcleo” con probabilidad acumulada p

0,9 por defecto



Top-k

Solo los k tokens más probables

40 es un valor común



Máx. tokens

Límite de longitud de la salida

Controla costo y latencia



Semilla

Mejora la reproducibilidad (si el proveedor la soporta)

Útil en pruebas



Stop

Cadenas que detienen la generación

Útil en formatos estructurados

Inferencia: prefill, decode y la ventana de contexto



Tiempo al primer token (TTFT)

Lo que el usuario percibe como “rapidez”. Crece con prompts largos.

Tokens por segundo

Velocidad de generación. Depende del tamaño del modelo y del hardware.

Ventana de contexto

Máximo de tokens entrada + salida. De ~8 mil a más de 1 millón según el modelo.

Limitaciones que todo arquitecto debe diseñar

Alucinaciones

Genera lo plausible, no lo verdadero

MITIGACIÓN

RAG · citas · validación · humano en el ciclo

Fecha de corte

Desconoce hechos recientes

MITIGACIÓN

Herramientas de búsqueda · RAG

No determinismo

Distintas salidas para la misma entrada

MITIGACIÓN

Temperatura baja · salidas estructuradas

Contexto finito

Documentos largos no caben

MITIGACIÓN

Fragmentación · resúmenes · recuperación

Sesgos

Refleja sesgos de sus datos

MITIGACIÓN

Evaluación representativa · guardarraíles

Prompt injection

Instrucciones maliciosas en la entrada

MITIGACIÓN

Separar instrucciones y datos · filtros

1.3

Relevancia de los LLM e impacto en la industria

Dónde encajan en la IA · Cambio de paradigma · Casos por sector en LATAM

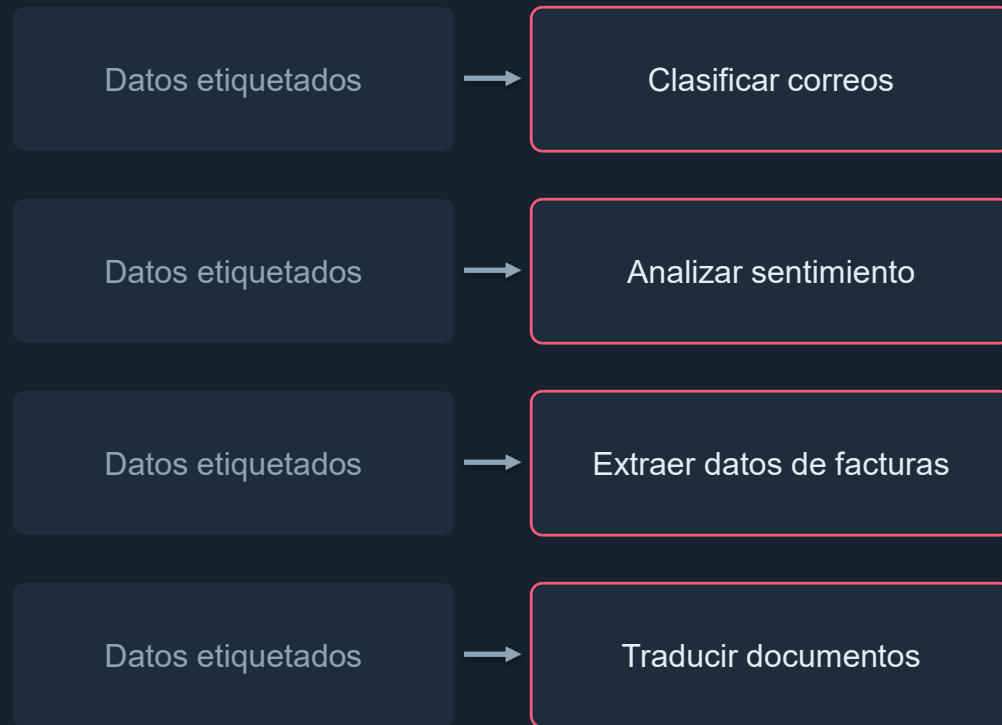
Dónde encajan los LLM dentro de la inteligencia artificial



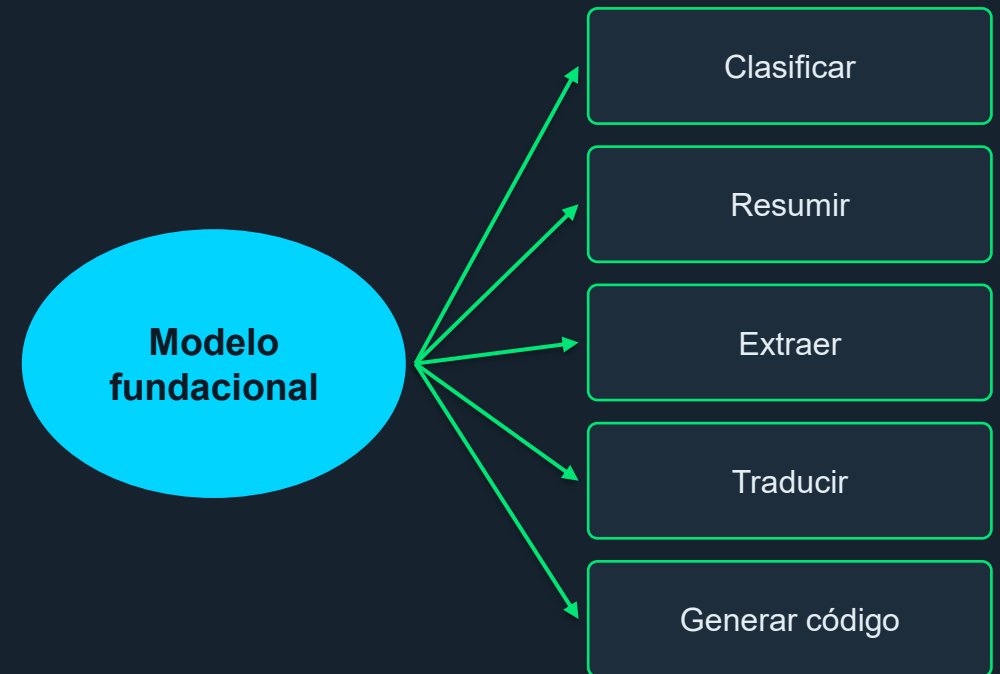
- **IA clásica:** scoring de crédito, fraude, pronóstico de demanda.
- **IA generativa:** crea contenido nuevo: texto, imagen, audio, código.
- **LLM:** IA generativa especializada en lenguaje; hoy también multimodal.
- **Criterio:** datos tabulares → ML clásico suele ser mejor y más barato; lenguaje no estructurado → LLM.

Modelos fundacionales: un cambio de paradigma para TI

ANTES · un modelo por tarea



AHORA · un modelo, muchas tareas



Se adapta con instrucciones, ejemplos, RAG o ajuste fino.

Impacto por sector: patrones de uso en Latinoamérica

F Financiero

Atención 24/7 · análisis documental
KYC · apoyo a cumplimiento

P Público

PQRS · orientación en trámites ·
lenguaje claro

S Salud

Resumen de historias clínicas · apoyo
a codificación CIE-10

M Manufactura

Mesa de ayuda · manuales técnicos ·
reportes de calidad

E Educación

Tutores personalizados ·
retroalimentación de escritura

C Comercio

Descripciones de producto · análisis de
reseñas

El LLM es un componente; la solución es la arquitectura



Del concepto a la evidencia (30 min)

01

Anatomía de una llamada API

Petición y respuesta HTTP · tokens · prefill vs. decode

02

Predicción del siguiente token

Distribución de probabilidad y temperatura (sin GPU)

03

Tokens, contexto y costo

Español vs. inglés · ventana de contexto · costo

04

Temperatura en un modelo real

Misma pregunta, distintas temperaturas

App

LLM Lab · Playground y Tokens

Streaming · métricas · visualización de tokens

PREPARACIÓN

- Ubuntu 24.04 o WSL2
- Python 3.12 y Node 22
- Ollama + llama3.2:3b
- Repositorio del curso
- **Plan B:** Ollama simulado

Qué entregar y cómo se evalúa

ENTREGABLE (1–2 PÁGINAS)

- 1 Capturas de los labs 01 y 04 con la interpretación de las métricas
- 2 Tabla: 3 frases de su organización, tokens en español vs. inglés
- 3 Argumento: ¿qué temperatura para extraer datos de facturas y por qué?

CRITERIOS DE EVALUACIÓN

Criterio	Peso
Explica la predicción del siguiente token y la atención	30 %
Interpreta métricas de su propia ejecución	30 %
Relaciona tokens y parámetros con costo y calidad	30 %
Claridad y evidencia	10 %

+ Quiz de 10 preguntas con retroalimentación

Tres ideas para llevarse

1

Predice, no consulta

Un LLM genera el siguiente token más probable. Hay que diseñar para sus errores.

2

Todo se mide en tokens

Costo, latencia y límite de contexto dependen de los tokens de entrada y salida.

3

El LLM es un componente

El valor y el riesgo están en la arquitectura que lo rodea.

Próxima sesión: tipos de modelos, nube vs. on-premise, casos LATAM y costos. Traiga un caso de uso de su organización.

02

Tipos de modelos y servicios

Arquitecturas · Nube y on-premise · Casos de uso LATAM · Costos y factores de selección

Al terminar esta sesión usted podrá...

Comprender

las arquitecturas decoder-only, encoder-only y encoder-decoder y sus usos

Comprender

los modelos de despliegue: API, nube gestionada, autohospedado, on-premise

Analizar

casos de uso de LLM en sectores de LATAM: datos, regulación y costo

Aplicar

la estimación de costo mensual y el punto de equilibrio de una solución

Evaluar

alternativas con una matriz de decisión ponderada y seleccionar la mejor

¿Un buscador semántico necesita generar texto?

BUSCAR POR SIGNIFICADO

“requisitos para un préstamo de vivienda”



Encoder → vector [0.12, -0.8, ...]



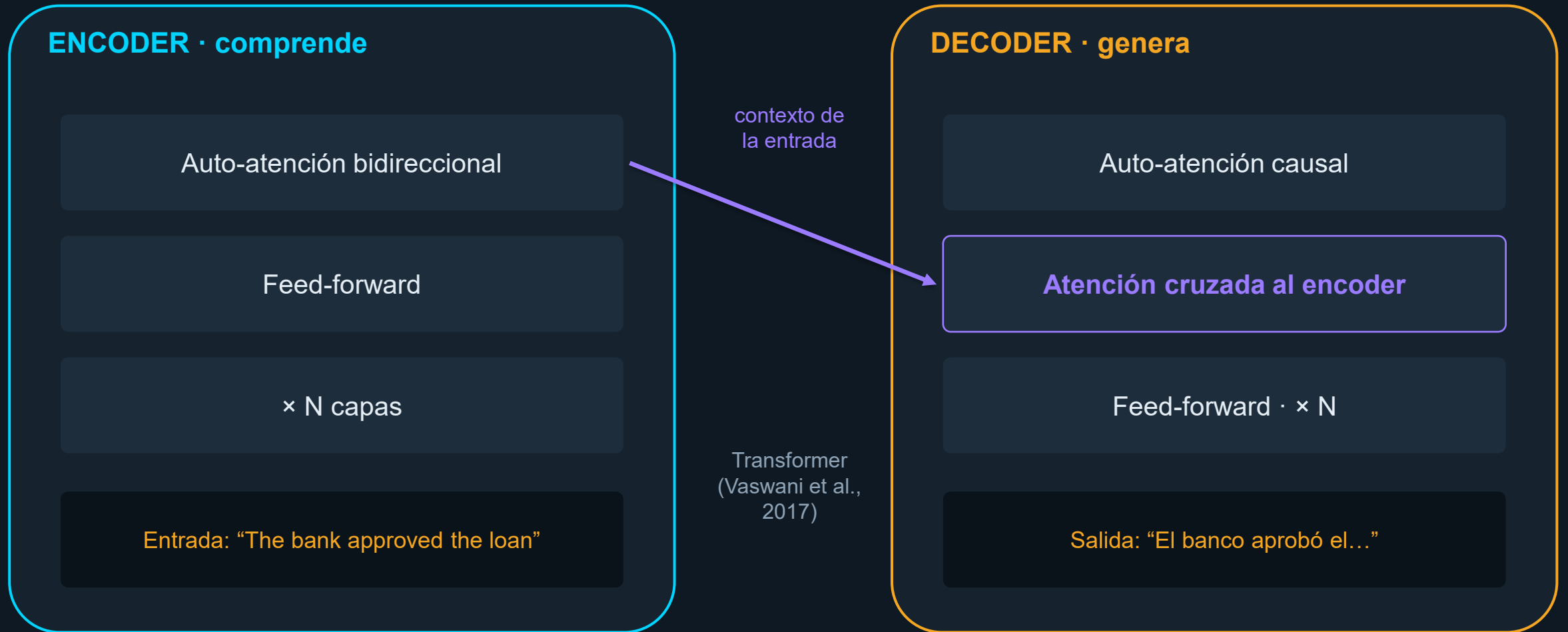
Documentos más cercanos:
“Crédito hipotecario: condiciones” · 0,91

LA IDEA DE HOY

Existen familias de modelos distintas, cada una optimizada para un tipo de tarea.

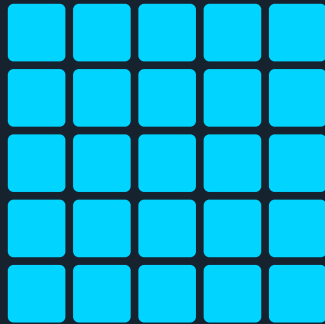
Elegir la familia correcta puede reducir el costo en órdenes de magnitud sin perder calidad.

Todo empieza en el Transformer original: encoder + decoder



Las tres familias: qué ve cada token

Encoder-only



ATENCIÓN

Bidireccional: ve toda la frase

PARA

Comprender · vector

BERT · modelos de embeddings

Decoder-only



ATENCIÓN

Causal: solo ve lo anterior

PARA

Generar · texto

GPT · Llama · Gemini · Claude

Encoder-decoder



ATENCIÓN

Encoder completo + decoder causal

PARA

Transformar · texto

T5 · BART · Whisper

Encoder-only: modelos que comprenden y representan

- **Entrenamiento:** adivinar palabras ocultas usando el contexto de ambos lados.
- **Salida:** un vector (embedding) por texto o por token.
- **Tamaño típico:** de decenas a cientos de millones de parámetros.
- **Despliegue:** corre en CPU; ideal on-premise y en tiempo real.

```
El [MASK] aprobó el crédito  
→ predice: banco (0,82) · comité (0,09)
```

1

Búsqueda semántica y RAG

2

Clasificación de PQRS e intenciones

3

Detección de duplicados

4

Anonimización (entidades)

Decoder-only: la familia que domina la IA generativa

1 Objetivo universal

Predecir el siguiente token funciona con cualquier texto, sin etiquetas.

2 Todo es “continuar texto”

Clasificar, resumir, traducir o programar con una instrucción.

3 Aprendizaje en contexto

Aprende tareas nuevas con ejemplos en el prompt, sin reentrenar.

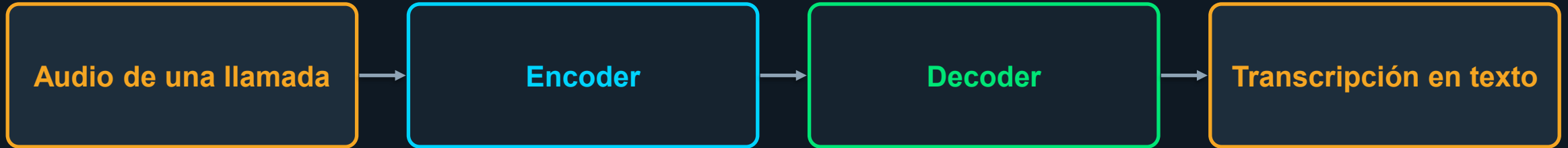
4 Escala

Las leyes de escala se demostraron sobre todo en esta familia.

LO QUE IMPLICA PARA EL ARQUITECTO

- Son los modelos más grandes y costosos
- La latencia crece con la longitud de la salida
- Se cobran por tokens de entrada y salida
- Pueden alucinar: requieren guardarraíles
- Muchos ya son multimodales

Encoder-decoder: transformar una secuencia en otra



Ejemplo: Whisper (voz → texto) en un centro de contacto

1 Traducción

Contratos y manuales entre español, portugués e inglés

2 Resumen

Documento largo → resumen ejecutivo

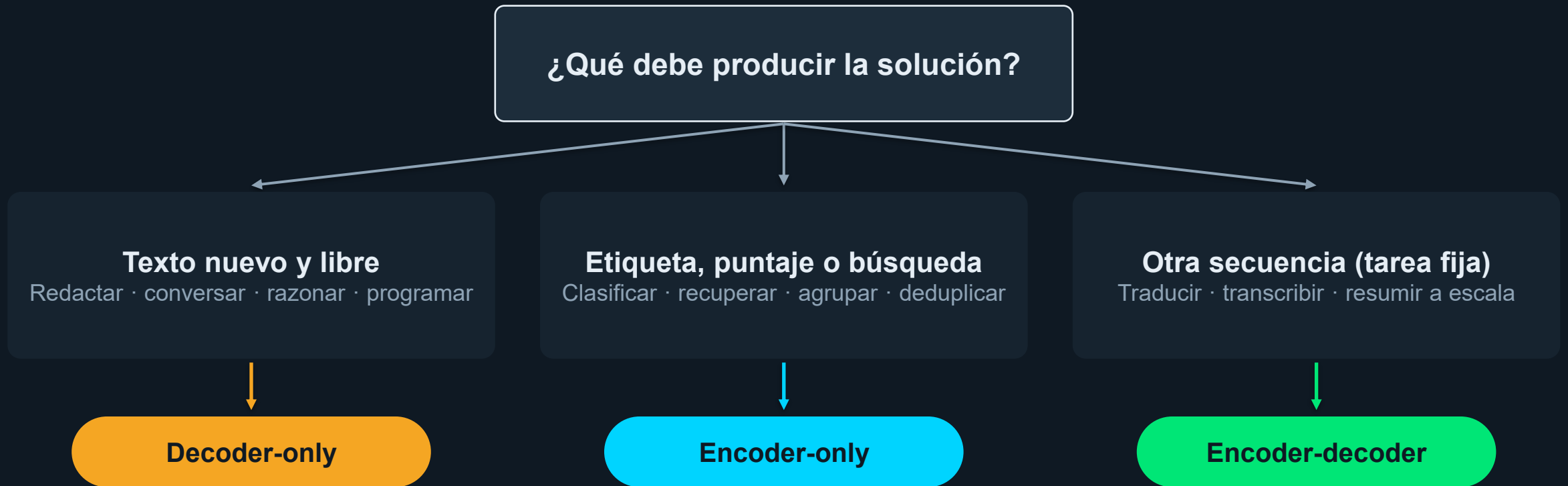
3 Voz a texto

Transcripción de llamadas y audiencias

4 Reescritura

Normas y cartas a lenguaje claro

¿Qué familia usar? Una guía de decisión



En la práctica se combinan: un encoder recupera y clasifica; un decoder redacta la respuesta (patrón RAG).

Variantes que conviene reconocer



Mezcla de expertos (MoE)

Muchos expertos, solo algunos activos por token: gran capacidad con menor costo de inferencia.



Modelos pequeños (SLM)

1–10 mil millones de parámetros. Corren en laptops o en el borde; ideales para tareas acotadas.



Multimodales

Entienden imágenes, audio o video: facturas escaneadas, formularios, fotos de inspección.



Modelos de razonamiento

“Piensan” en pasos antes de responder: más calidad en problemas complejos, más tokens y latencia.

Encoder + decoder juntos: anticipo del patrón RAG



El encoder encuentra la información; el decoder la explica. Así se reducen las alucinaciones y se usa conocimiento propio y actualizado sin reentrenar el modelo.

2.2

Servicios en la nube y on-premise

Espectro de despliegue · Pesos abiertos vs. cerrados · Arquitecturas híbridas · Interoperabilidad

El espectro de despliegue: conveniencia vs. control

Más conveniencia

Más control y soberanía

1

API del fabricante

OpenAI · Anthropic
Gemini API

Cero infraestructura · pago
por uso · modelos de frontera

2

Nube gestionada

Azure AI Foundry · AWS
Bedrock · Agent Platform

Contrato empresarial · red
privada · IAM · SLA

3

Autohospedado en nube

vLLM · TGI en GPU
o Kubernetes

Control del modelo y de los
datos en su nube

4

On-premise / borde

Ollama · llama.cpp
vLLM en servidores propios

Máxima soberanía · sin costo
por token

Pesos abiertos vs. cerrados

Cerrados

Solo por API

- Suelen liderar en capacidades
- El proveedor opera todo
- Dependencia del proveedor
- Versiones fuera de su control

GPT · Claude · Gemini

Pesos abiertos

Se descargan y ejecutan donde quiera

- Control y privacidad
- Ajuste fino con datos propios
- Usted opera la infraestructura
- Los más capaces requieren GPU costosas

Llama · Mistral · Gemma · Qwen · DeepSeek

¡Ojo con la licencia!

“Abierto” no significa “libre para cualquier uso”.

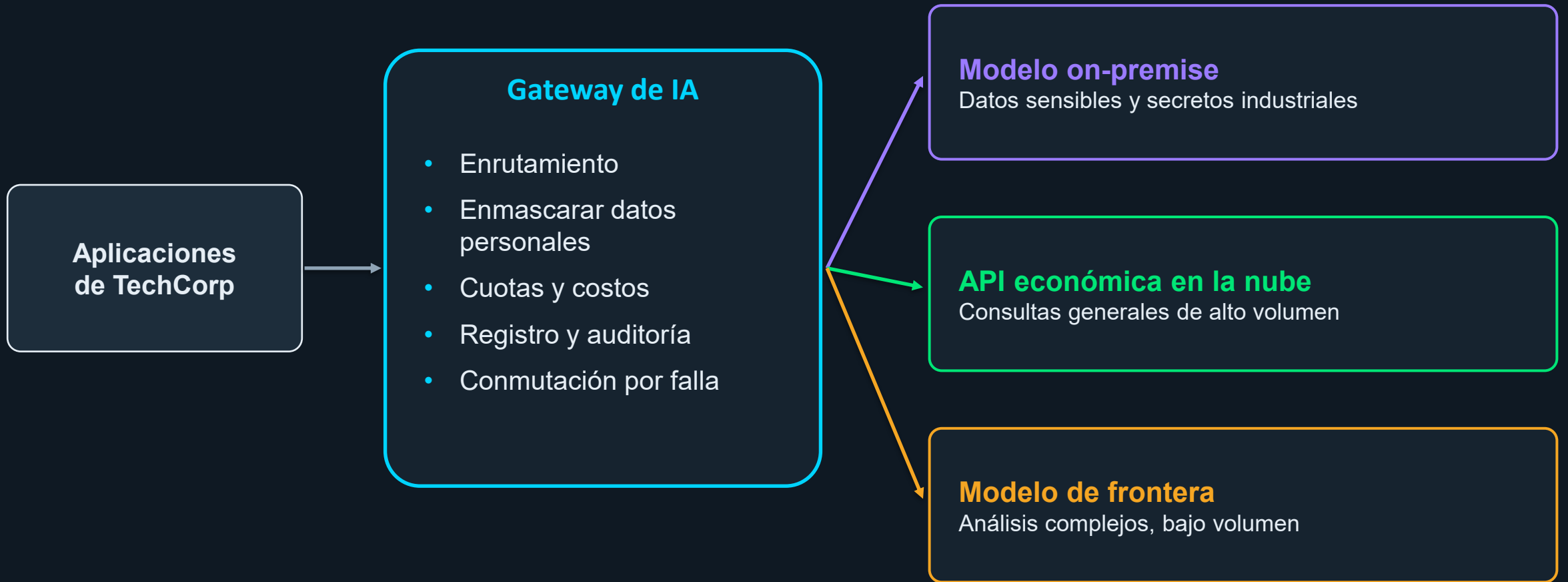
Algunas licencias restringen el uso comercial, el número de usuarios o ciertos casos de uso.

Revisión legal antes de producción.

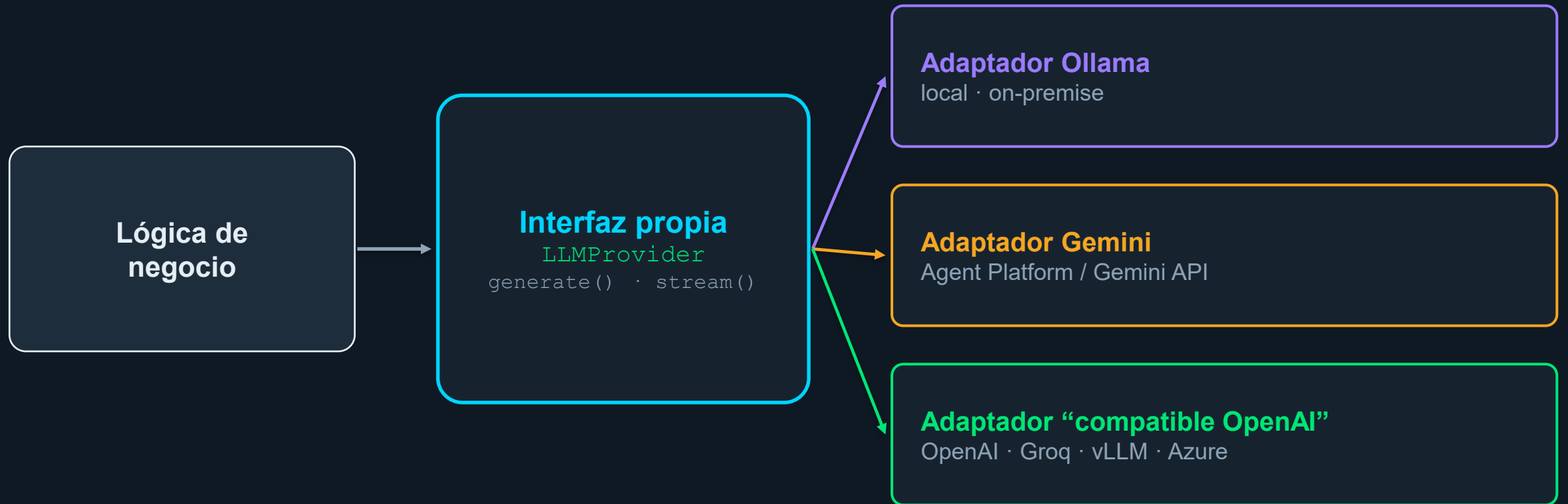
Nube gestionada vs. on-premise: comparación por criterio

Criterio	Nube gestionada (API / hiperescalador)	On-premise / autohospedado
Tiempo de salida al mercado	Días	Semanas a meses
Soberanía de datos	Media–alta (región, red privada, contrato)	Máxima
Modelo de costo	Variable: pago por token	Fijo: hardware + operación
Escalabilidad	Casi ilimitada y elástica	Limitada por el hardware
Acceso a modelos de frontera	Sí	Solo pesos abiertos
Operación requerida	Baja	Alta (MLOps, GPU, seguridad)
Latencia	Depende de la red y la región	Baja en la red local

Arquitectura híbrida: un gateway que enruta según el caso



Interoperabilidad: una interfaz propia, varios proveedores



Cambiar de proveedor = cambiar configuración, no reescribir el negocio. (Ver backend/app/providers en el repositorio.)

2.3

Casos de uso en Latinoamérica

Sector público · Sector financiero · Salud · Regulación y residencia de datos

Sector público: casos, arquitectura sugerida y consideraciones

PQRS ciudadanas

1

Clasificar y enrutar solicitudes a la dependencia correcta

Encoder + decoder

Orientación en trámites

2

Asistente sobre requisitos, costos y pasos

Decoder + RAG sobre normativa

Lenguaje claro

3

Reescribir actos administrativos para la ciudadanía

Decoder

CONSIDERACIONES

- Transparencia y trazabilidad de las decisiones
- Accesibilidad e inclusión (lenguas y regiones)
- Presupuesto público: priorizar costo por solicitud

Sector financiero: casos, arquitectura sugerida y consideraciones

Atención al cliente

1

Asistente 24/7 sobre productos y trámites

Decoder en nube
gestionada

Análisis documental

2

Extraer datos de documentos para conocimiento del cliente

Decoder multimodal

Apoyo a cumplimiento

3

Resumir alertas y expedientes para analistas

Decoder + RAG

CONSIDERACIONES

- Datos personales y secreto bancario
- Supervisión financiera: auditoría y explicabilidad
- Humano en el ciclo para decisiones de crédito o fraude

Salud: casos, arquitectura sugerida y consideraciones

Resumen clínico

1

Resumir historias clínicas extensas para el profesional

Decoder en entorno privado

Apoyo a codificación

2

Sugerir códigos CIE-10 para validación

Encoder + decoder

Gestión y educación

3

Citas, recordatorios y material para pacientes

Decoder

CONSIDERACIONES

- Datos sensibles: consentimiento, anonimización, residencia
- Validación clínica obligatoria
- Sesgos en poblaciones subrepresentadas

Regulación y residencia de datos en la región

Colombia	Ley 1581 de 2012 · política de IA (CONPES)
Brasil	LGPD · Ley 13.709 de 2018
Chile	Ley 21.719 de 2024
México	LFPDPPP
Perú	Ley 29733
Argentina	Ley 25.326

Referencia: verifique la norma vigente en cada país. No es asesoría legal.

Residencia de datos

Regiones locales existen, pero no todos los modelos están en todas.

Transferencias

Enviar datos personales a otro país puede requerir bases legales.

Minimización

Anonimizar o enmascarar antes de enviar al modelo.

Trazabilidad

Registrar llamadas para auditorías y derechos de los titulares.

2.4

Comparativa de costos y factores de selección

Cómo se cobra · Ejemplo de cálculo · Punto de equilibrio · TCO · Matriz de decisión

Cómo se cobra un LLM

tokens de entrada

× precio entrada

+

tokens de salida

× precio salida

+

costo fijo

si es autohospedado

4–8×

más cuesta un token de salida que uno de entrada (típico)

USD / 1M

unidad de precio habitual: dólares por millón de tokens

Caché y lotes

descuentos por reutilizar contexto o procesar de forma asíncrona

Ejemplo: chatbot bancario de 20.000 conversaciones diarias

600 mil

solicitudes al mes

870 M

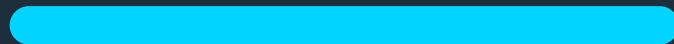
tokens al mes (1.200 in + 250 out)

USD 591

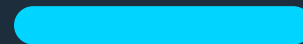
al mes con un modelo tipo “flash”
(ilustrativo)

¿DÓNDE ESTÁ EL COSTO?

Entrada



83 % de los tokens



37 % del costo

Salida



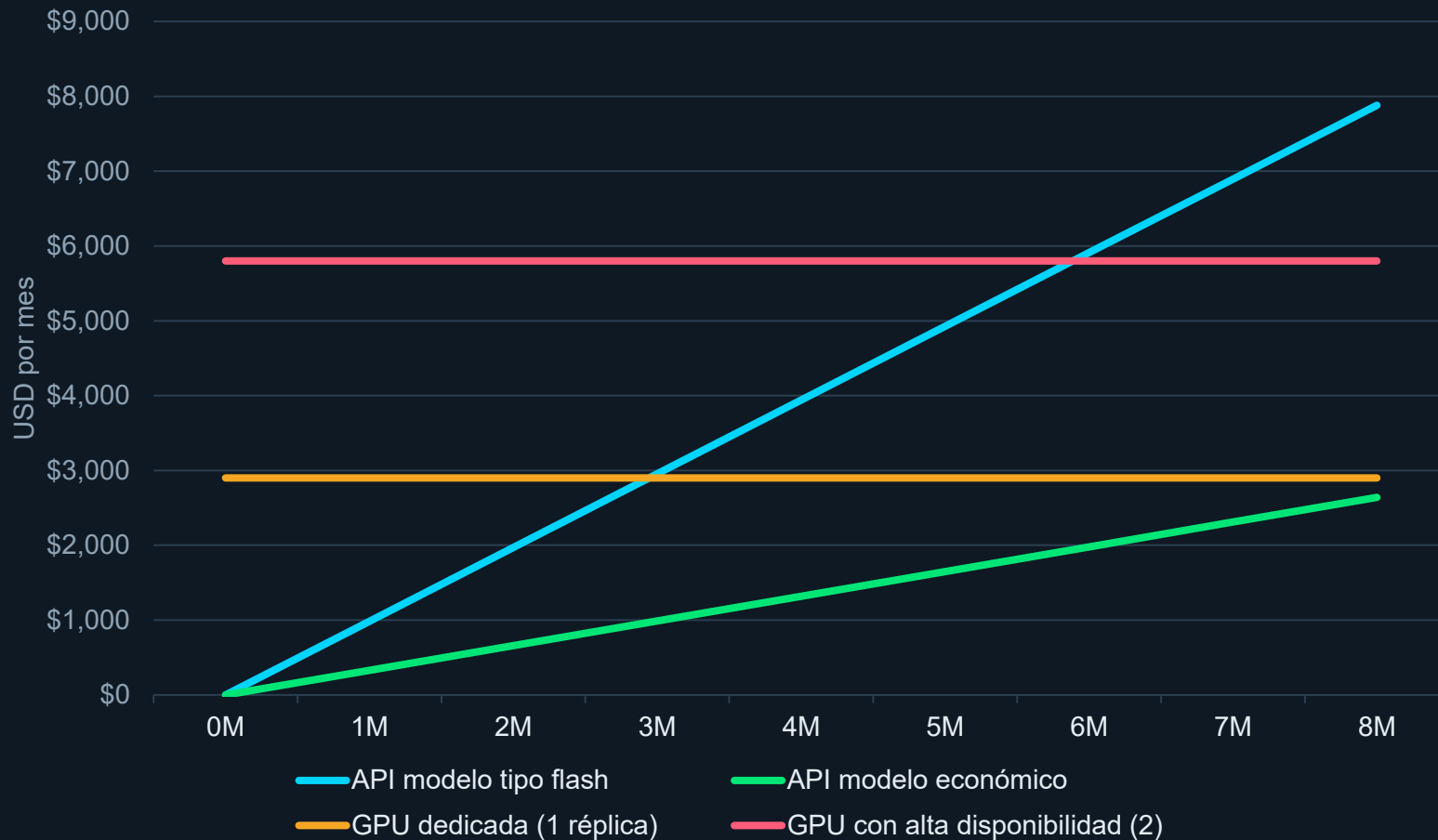
17 % de los tokens



63 % del costo

La salida es poca... pero es la más cara

Pago por uso vs. infraestructura propia: el punto de equilibrio



Eje horizontal: solicitudes al mes (millones) · precios ilustrativos



Equilibrio

≈ 2,9 M solicitudes/mes frente al modelo tipo flash (≈ 98 mil por día).

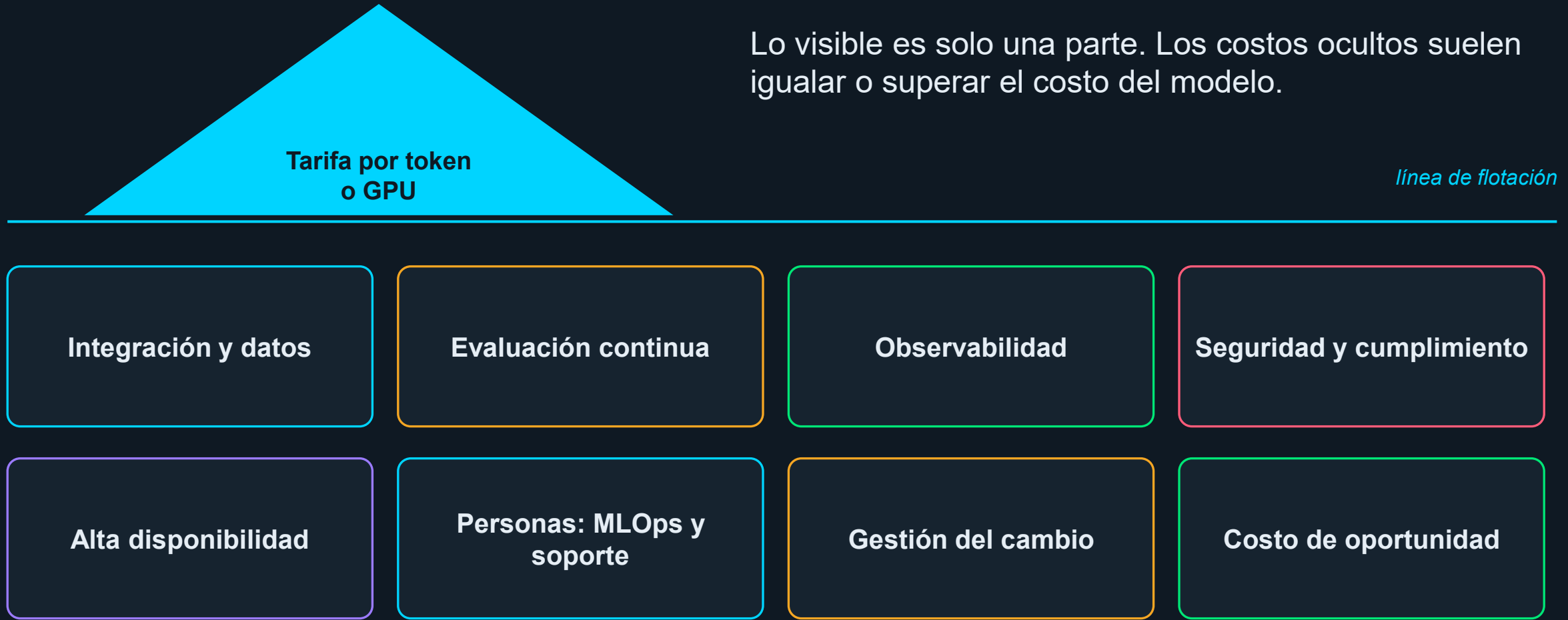


Lo que falta

Operación, capacidad máxima de la GPU y diferencia de calidad.

Costo total de propiedad: lo que no aparece en la tarifa

Lo visible es solo una parte. Los costos ocultos suelen igualar o superar el costo del modelo.



Tarifa por token
o GPU

línea de flotación

Integración y datos

Evaluación continua

Observabilidad

Seguridad y cumplimiento

Alta disponibilidad

Personas: MLOps y
soporte

Gestión del cambio

Costo de oportunidad

Palancas de optimización de costos

1 Modelo adecuado

El más pequeño que cumpla la calidad; enrutar solo lo difícil al grande.

2 Controlar la salida

Límite de tokens, respuestas concisas, JSON en vez de prosa.

3 Caché

De prompts (proveedor) y de respuestas frecuentes (propia).

4 Menos contexto

RAG en lugar de documentos completos; resumir historiales.

5 Lotes asíncronos

Procesos no interactivos con APIs batch con descuento.

6 Medir por caso de uso

Sin observabilidad de costo no hay optimización.

Factores de selección de un LLM

1

Calidad en su tarea

2

Costo a escala

3

Latencia

4

Privacidad y residencia

5

Cumplimiento

6

Contexto y capacidades

7

Calidad en español

8

Operación, SLA y soporte

9

Licencia y dependencia

Matriz de decisión ponderada: ejemplo para el sector financiero

Opción	Calidad 20%	Costo 10%	Latencia 10%	Privacidad 20%	Cumplim. 25%	Operación 5%	Español 10%	Puntaje
API gama alta	5	2	3	2	3	5	5	3,40
API económica	3	5	4	2	3	5	4	3,30
Abierto en nube privada	4	3	4	4	4	3	4	3,85
Abierto pequeño on-prem	2	4	3	5	5	2	3	3,75

Gana: Abierto en nube privada (3,85). Con los pesos de una startup de comercio, gana la API económica.
Calificaciones ilustrativas para discusión.

De la teoría a la decisión (45 min)

01

Encoder vs. decoder

Misma tarea: exactitud, latencia y formato

02

Mismo código, varios proveedores

Interoperabilidad: solo cambia la configuración

03

Punto de equilibrio

API de pago por uso vs. GPU dedicada

04

Matriz de decisión

Pesos por sector y análisis de sensibilidad

App

LLM Lab · pestañas 3, 4 y 5

Embeddings · comparador · calculadora de costos

Un mismo agente, tres formas de construirlo

1 Consola

Agent Studio · sin código

→ Agent Runtime

- Prototipos rápidos
- Usuarios de negocio
- Menor control

2 Google ADK

agent.py + herramientas Python

→ Agent Runtime

- Sesiones gestionadas
- Código abierto y portable
- Buen equilibrio

3 Sin ADK

FastAPI + google-genai · Docker

→ Cloud Run

- Control total
- Máxima portabilidad
- Usted opera todo

Memo de recomendación (máx. 2 páginas)

- 1 Caso y datos involucrados
- 2 Familia de modelo recomendada
- 3 Modelo de despliegue y regulación
- 4 Costo mensual con supuestos
- 5 Matriz de decisión y sensibilidad
- 6 Riesgos y mitigaciones

Criterio	Peso
Justifica la familia de modelo según la tarea	25 %
Elige el despliegue considerando datos, regulación y operación	25 %
Estima costos con supuestos explícitos	25 %
Matriz coherente con el sector y análisis de riesgos	25 %

+ Quiz de 10 preguntas con retroalimentación

Lo que un arquitecto de soluciones LLM ya sabe

1

Predice tokens

Diseño para sus limitaciones: alucinaciones, contexto, costo.

2

Familias distintas

Encoder comprende · decoder genera · encoder-decoder transforma.

3

Espectro de despliegue

De la API al on-premise; el híbrido con gateway combina ambos.

4

Decisión cuantificada

Tokens, punto de equilibrio, TCO y matriz ponderada.

Próximo capítulo: consumir un endpoint paso a paso, y optimización de costos y rendimiento.

Lecturas recomendadas

- Vaswani et al. (2017). Attention Is All You Need. NeurIPS.
- Devlin et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers.
- Radford et al. (2019). Language Models are Unsupervised Multitask Learners (GPT-2).
- Raffel et al. (2019). Exploring the Limits of Transfer Learning... (T5).
- Brown et al. (2020). Language Models are Few-Shot Learners (GPT-3).
- Kaplan et al. (2020). Scaling Laws for Neural Language Models.
- Hoffmann et al. (2022). Training Compute-Optimal LLMs (Chinchilla).
- Ouyang et al. (2022). Training language models to follow instructions with human feedback.
- Bommasani et al. (2021). On the Opportunities and Risks of Foundation Models.
- J. Alammam. The Illustrated Transformer (blog).
- Documentación: Ollama · Google Gemini Enterprise Agent Platform · Google ADK.
- Repositorio del curso: guías, laboratorios y evaluación.

¿Preguntas?

Fundamentos de Arquitectura LLM · Capítulo 1

Repositorio: guías, laboratorios, LLM Lab y laboratorio de Google Cloud

